

cup-viewer — страница согласования макета

Клиент типографии открывает персональную ссылку и видит свой будущий стакан в 3D: может покрутить его, рассмотреть макет плоско на вкладке «Развёртка», согласовать или запросить правки.

Для IT-отдела — как развернуть и как подключить к Битриксу: [docs/integration.md](#) .

Чего пока нет. Кнопка «Согласовать» и форма правок ничего не отправляют на сервер: бэкенда для них не существует. Что нужно дописать — в [docs/integration.md](#) , раздел «Что осталось сделать».

1. Быстрый старт

```
npm install
npm start
```

Откроется <http://localhost:8000>.

Двойным кликом по `index.html` страница не заработает: браузер запрещает JS-модулям и 3D-файлам грузиться с диска, нужен HTTP-сервер.

2. Где что править

Главная таблица этого файла. Если непонятно, куда лезть за изменением, — сюда.

Хочу изменить	Файл
Текст, зависящий от заказа: статус, заголовок, описание, характеристики, пункты чек-листа	<code>js/data/order.js</code> → <code>DEFAULT_ORDER</code>
Любой другой текст: подписи кнопок, вкладки, подсказки, заголовок вкладки браузера	<code>index.html</code>
Цвета, отступы, шрифты, поведение на телефоне	<code>css/approve.css</code>
Свет, камера, фоны, шероховатость бумаги, размер тени	<code>js/config.js</code>

Хочу изменить	Файл
Какая 3D-модель под какую номенклатуру, шероховатость картона	<code>js/data/catalog.js</code>
Иконки панели	заменить файл в <code>assets/icons/</code> под тем же именем
3D-модель, макет, карта освещения	<code>assets/models/</code> , <code>assets/textures/</code> , <code>assets/hdri/</code>

После правки любого файла в `js/` нужно выполнить `npm run build` . Браузер читает не исходники, а собранный `dist/cup-viewer.js` — почему так, см. раздел 5. Чтобы не запускать сборку руками при каждой правке:

```
npm run watch
```

Почему текст разложен на две части

Раньше он был продублирован: одни и те же строки лежали и в `index.html` , и в JS. Правка в разметке ничего не давала — JS перезаписывал её при загрузке, и в этот момент успевала мелькнуть старая версия. Теперь правило жёсткое:

- **зависит от заказа** (у каждого клиента своё) → приходит из Битрикса, а умолчания лежат в `js/data/order.js` . В `index.html` эти места **пустые**, и писать туда текст бесполезно;
- **не зависит от заказа** (одинаково всегда) → прямо в `index.html` .

Подписи кнопки согласования — тоже в разметке, в `data-label-*` на самой кнопке: JS только переключает состояние, а текст не придумывает.

3. Как устроена страница

```
index.html → dist/cup-viewer.js (собранный JS)
|
├─ js/main.js ..... точка входа: поднимает сцену, грузит заказ
|   └─ js/data/order.js ..... собирает заказ из всех источников
|   └─ js/ui/orderPanel.js .. заполняет правую панель
|       └─ js/Viewer.js ..... 3D-сцена целиком
|
└─ js/ui/approval.js .. вкладки, чек-лист, форма правок, подсказка
```

Откуда берутся данные заказа

Приоритет — сверху вниз, что выше, то важнее:

Источник	Когда используется
Параметры адреса <code>?sku=...&texture=...</code>	только на <code>localhost</code> , для проверки руками
<code>window.ORDER_DATA</code>	шаблон Битрикса печатает объект прямо в страницу — основной способ
Запрос на сервер	если задан <code>config.order.endpoint</code> и в адресе есть <code>?order=...</code>
<code>DEFAULT_ORDER</code> в <code>js/data/order.js</code>	всё остальное, когда данных не пришло

Всё, что пришло снаружи, проходит проверку в `js/data/sanitize.js`: незнакомые поля выбрасываются, длинные строки режутся, а путь к файлу принимается только свой — чужой домен или `javascript:` не пройдут. Страница обязана открыться в любом случае, поэтому непрошедшее поле молча заменяется значением по умолчанию, а причина пишется в консоль браузера.

Что делает `js/Viewer.js`

Это «дирижёр» 3D-сцены. Он собирает вместе части из `js/core/` и владеет циклом отрисовки:

Файл	За что отвечает
<code>core/scene.js</code> , <code>core/camera.js</code>	сцена и камера — по три строки каждый
<code>core/renderer.js</code>	вывод картинки и цветовой конвейер
<code>core/environment.js</code>	загрузка карты освещения, поиск в ней солнца
<code>core/lighting.js</code>	источники света (в текущей настройке почти не используются)
<code>core/contactShadow.js</code>	мягкая тень под стаканом
<code>core/controls.js</code>	вращение камеры мышью
<code>core/turntable.js</code>	поворотный круг под стаканом
<code>loaders/ModelLoader.js</code>	загрузка <code>.glb</code> и подгонка модели под кадр
<code>loaders/TextureManager.js</code>	загрузка макета и наложение его на модель
<code>ui/UIManager.js</code>	кнопки панели и горячие клавиши

4. Ключевые решения — почему сделано именно так

Раздел для того, чтобы код не выглядел магией.

Свет берётся из фотографии неба, а не из ламп

В `assets/hdri/main.hdr` лежит панорама реального помещения с полным диапазоном яркости. Такая карта освещает модель со всех сторон сразу — стакан выглядит как сфотографированный, а не как подсвеченный тремя лампочками. Это стандартный приём, называется IBL (image-based lighting).

Карта используется **только как источник света**, а не как фон: фон остаётся плоским управляемым цветом, который клиент переключает кружочками.

Разрешение карты специально снижено до 512×256 (512 КБ вместо 6,8 МБ). Перед использованием она всё равно размывается движком, поэтому на картинке разницы нет, а страница открывается заметно быстрее. Пережать другую карту:

```
node scripts/resize-hdri.mjs assets/hdri/main.hdr      # показать варианты
node scripts/resize-hdri.mjs assets/hdri/main.hdr 512 --write
```

Тень нарисована, а не рассчитана

Обычные 3D-тени часто дают артефакты: полосы, отрыв тени от предмета, засветы. Здесь тень делается иначе: модель рендерится сверху в отдельную картинку, картинка размывается и кладётся на пол под стаканом. Никаких артефактов принципиально быть не может, настраивать нечего.

Направление, мягкость и плотность тени берутся из самой карты освещения: код находит на панораме самое яркое пятно (солнце или окно) и разворачивает тень туда же. Поэтому тень выглядит согласованной со светом.

Посмотреть, что нашлось — в консоли браузера: `cupViewer.describeSun()` .

Вращается модель, а не камера

При автоповороте крутится сам стакан, камера стоит. Если бы вращалась камера, вместе с ней «поехали» бы блики и отражения — стакан выглядел бы неестественно. При перетаскивании мышью, наоборот, вращается камера: это привычный облёт.

Кадр рисуется не всегда

Пока на экране ничего не меняется (клиент читает характеристики, смотрит развёртку), сцена не перерисовывается. Иначе телефон 60 раз в секунду рисовал бы одну и ту же картинку и грелся. Свёрнутая вкладка останавливает отрисовку полностью.

Если добавляете что-то, что меняет сцену, — вызовите `viewer.invalidate()` (или `viewer.invalidateModel()` , если сдвинулась сама модель: тогда пересчитается ещё и тень). Без этого изменение может не появиться на экране.

Макет уменьшается при загрузке

Макеты приходят в печатном разрешении — 7000 пикселей по ширине норма. Видеокарта телефона такую картинку либо не примет вовсе, либо займёт под неё почти всю доступную память. Поэтому на модель кладётся уменьшенная копия (потолок — `config.texture.maxSize` , по умолчанию 2048 px).

Для вкладки «Развёртка», где смотрят мелкий текст, можно передать отдельным полем `unwrap` крупный исходник — он показывается плоско и на видеокарту не попадает.

Развёртка рисуется в canvas

Простая картинка при увеличении показывала бы не детали макета, а увеличенные пиксели экрана. Здесь макет каждый раз перерисовывается из исходного файла в физических пикселях устройства, поэтому приближение упирается только в разрешение самого файла.

5. Зачем нужна сборка

Браузер загружает один файл `dist/cup-viewer.js` , а не два десятка файлов из `js/` . Три причины:

1. **Библиотека three.js лежит внутри нашего кода.** Раньше она грузилась со стороннего сайта (unpkg.com) — если тот недоступен, страница не открывается вовсе, а если его подменяют, на странице клиента исполнится чужой код.
2. **Совместимость.** Прежний способ подключения модулей не поддерживает Safari старше версии 16.4 — на таких iPhone страница не открывалась и не сообщала об этом.
3. **Обновления доезжают до клиента.** Сборка дописывает к адресам файлов метку версии (`?v=a1b2c3d4`), и браузер не отдаёт закешированную старую версию.

`dist/` лежит в репозитории, поэтому **на сервере Node не нужен** — достаточно скопировать файлы. Node нужен только тому, кто правит `js/` .

Команда	Что делает
<code>npm run build</code>	собрать один раз
<code>npm run watch</code>	пересобирать при каждой правке
<code>npm run serve</code>	только локальный сервер
<code>npm start</code>	собрать и запустить

6. Управление сценой

Мышь: левая кнопка — облёт камерой, колесо — зум, правая — сдвиг, двойной клик — вернуть исходный вид. Круг под стаканом вращает сам стакан, свет остаётся на месте.

Действие	Кнопка	Клавиша
Автоповорот	стрелки по кругу	A
Снимок сцены	камера	S
Во весь экран	рамка	F
Сменить фон	кружочки справа сверху	B
Поворотный круг	круг	—
Как управлять	«i»	—

Горячие клавиши молчат, пока курсор стоит в поле ввода, — иначе буква, набранная в форме правок, запускала бы скачивание снимка.

7. Настройки в `js/config.js`

Файл прокомментирован построчно; ниже — то, что трогают чаще всего.

Шероховатость бумаги

Главная настройка внешнего вида. У материала под макетом один параметр:

```
texturedSurface: {
  roughness: 0.8, // 0 — зеркало, 1 — полностью матовая
}
```

Значение	Как выглядит
0.2	мелованная бумага, лак — собранный блик
0.8	немелованная, обычная печать — рассеянный свет

Обычно значение приходит из заказа по типу картона (`js/data/catalog.js`), а не берётся отсюда.

Приём перетаскиваемых файлов

```
ui: { dragAndDrop: false }
```

Выключено намеренно. На клиентской странице брошенный на неё файл подменил бы согласуемый макет, и кнопка «Согласовать» зафиксировала бы не то, что прислала типография. Включать только для отладки.

Снимок сцены

```
ui: { screenshotShare: 'auto' }
```

`auto` — скачивать файлом. Системное окно «Поделиться» открывается только на iPhone и iPad, где Safari иначе не даёт сохранить картинку.

Безопасность

```
security: {  
  urlOverrides: 'localhost', // 'never' | 'localhost' | 'always'  
  allowedAssetPrefixes: ['assets/', '/upload/'],  
  debugApi: 'localhost',  
}
```

`urlOverrides` — разрешить ли параметрам адреса подменять данные заказа. На бою это опасно: по ссылке `?texture=...&title=Заказ+аннулирован` клиенту можно показать чужой макет и чужой текст на вашем же домене. Поэтому по умолчанию канал открыт только на локальной машине.

8. Проверка из консоли браузера

Работает на `localhost` (управляется `security.debugApi`):

```
cupViewer.order // какие данные применились  
cupViewer.applyOrder({ paper: 'coated' }) // подменить тип картона на лету  
cupViewer.applyOrder({ sku: 'DW90-430' }) // другая модель  
cupViewer.setRoughness(0.35) // покрутить шероховатость руками  
cupViewer.describeSun() // что нашлось в карте освещения
```

9. Структура файлов

index.html	разметка страницы и весь текст, не зависящий от заказа
dist/cup-viewer.js	собранный код – то, что читает браузер
css/style.css	базовые стили просмотрщика
css/approve.css	оформление страницы и адаптив под телефоны
js/config.js	настройки: камера, свет, тень, фоны, безопасность
js/main.js	точка входа
js/appState.js	связь между сценой и интерфейсом
js/Viewer.js	3D-сцена, цикл отрисовки
js/core/	части сцены: рендерер, свет, тень, камера, круг
js/loaders/	загрузка модели и макета
js/data/order.js	данные заказа и умолчания текста
js/data/catalog.js	номенклатура → модель, картон → шероховатость
js/data/sanitize.js	проверка данных, пришедших извне
js/ui/	панель управления, вкладки, развёртка, согласование
scripts/	сборка, локальный сервер, пережатие карты освещения
vendor/draco/	распаковщик сжатых 3D-моделей (своя копия)
assets/	модели, макеты, карта освещения, иконки
docs/integration.md	для IT-отдела: развёртывание и подключение к Битриксу
tools/illustrator/	отдельный скрипт цветопробы, к сайту не относится